



中山大學
SUN YAT-SEN UNIVERSITY

《计算机组成原理实验》 实验报告

(实验二)

学 院 名 称 : 计算机学院
专业 (班级) : 22 计科教学 3 班
学 生 姓 名 :
学 号 :
时 间 : 2023 年 1 月 22 日

成绩：

实验二：单周期CPU设计与实现

一. 实验目的

- (1) 掌握单周期 CPU 数据通路图的构成、原理及其设计方法；
- (2) 掌握单周期 CPU 的实现方法，代码实现方法；
- (3) 认识和掌握指令与 CPU 的关系；
- (4) 掌握测试单周期 CPU 的方法。

二. 实验内容

设计一个单周期 CPU，该 CPU 至少能实现以下指令功能操作。指令与格式如下：

==> 算术运算指令

(1) add rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100000
--------	---------	---------	---------	--------------

功能：GPR[rd] $\leftarrow$ GPR[rs] + GPR[rt]。

(2) sub rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100010
--------	---------	---------	---------	--------------

功能：GPR[rd] $\leftarrow$ GPR[rs] - GPR[rt]。

(3) addiu rt, rs, immediate

001001	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能：GPR[rt] $\leftarrow$ GPR[rs] + sign_extend(immediate); immediate 做符号扩展再参加“与”运算。

==> 逻辑运算指令

(4) andi rt, rs, immediate

001100	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能：GPR[rt] $\leftarrow$ GPR[rs] and zero_extend(immediate); immediate 做 0 扩展再参加“与”运算。

(5) and rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100100
--------	---------	---------	---------	--------------

功能：GPR[rd] $\leftarrow$ GPR[rs] and GPR[rt]。

(6) ori rt, rs, immediate

001101	rs(5 位)	rt(5 位)	immediate(16 位)	
--------	---------	---------	-----------------	--

功能：GPR[rt] $\leftarrow$ GPR[rs] or zero_extend(immediate)。

(7) or rd, rs, rt

000000	rs(5 位)	rt(5 位)	rd(5 位)	00000 100101
--------	---------	---------	---------	--------------

功能：GPR[rd] $\leftarrow$ GPR[rs] or GPR[rt]。

==>移位指令

(8) sll rd, rt, sa

000000	00000	rt(5 位)	rd(5 位)	sa(5 位)	000000
--------	-------	---------	---------	---------	--------

功能: $\text{GPR}[\text{rd}] \leftarrow \text{GPR}[\text{rt}] \ll \text{sa}$ 。

==>比较指令

(9) slti rt, rs, **immediate** 带符号数

001010	rs(5 位)	rt(5 位)	immediate (16 位)
--------	---------	---------	-------------------------

功能: if $\text{GPR}[\text{rs}] < \text{sign_extend}(\text{immediate})$ $\text{GPR}[\text{rt}] = 1$ else $\text{GPR}[\text{rt}] = 0$ 。

==> 存储器读/写指令

(10) sw rt, **offset** (rs) 写存储器

101011	rs(5 位)	rt(5 位)	offset (16 位)
--------	---------	---------	----------------------

功能: $\text{memory}[\text{GPR}[\text{base}] + \text{sign_extend}(\text{offset})] \leftarrow \text{GPR}[\text{rt}]$ 。

(11) lw rt, **offset** (rs) 读存储器

100011	rs(5 位)	rt(5 位)	offset (16 位)
--------	---------	---------	----------------------

功能: $\text{GPR}[\text{rt}] \leftarrow \text{memory}[\text{GPR}[\text{base}] + \text{sign_extend}(\text{offset})]$ 。

==> 分支指令

(12) beq rs, rt, **offset**

000100	rs(5 位)	rt(5 位)	offset (16 位)
--------	---------	---------	----------------------

功能: if($\text{GPR}[\text{rs}] = \text{GPR}[\text{rt}]$) $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$

else $\text{pc} \leftarrow \text{pc} + 4$

特别说明: **offset** 是从 PC+4 地址开始和转移到的指令之间指令条数。**offset** 符号扩展之后左移 2 位再相加。为什么要左移 2 位? 由于跳转到的指令地址肯定是 4 的倍数(每条指令占 4 个字节), 最低两位是“00”, 因此将 **offset** 放进指令码中的时候, 是右移了 2 位的, 也就是以上说的“指令之间指令条数”。

(13) bne rs, rt, **offset**

000101	rs(5 位)	rt(5 位)	offset (16 位)
--------	---------	---------	----------------------

功能: if($\text{GPR}[\text{rs}] \neq \text{GPR}[\text{rt}]$) $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$

else $\text{pc} \leftarrow \text{pc} + 4$

(14) bltz rs, **offset**

000001	rs(5 位)	00000	offset (16 位)
--------	---------	-------	----------------------

功能: if($\text{GPR}[\text{rs}] < 0$) $\text{pc} \leftarrow \text{pc} + 4 + \text{sign_extend}(\text{offset}) \ll 2$

else $\text{pc} \leftarrow \text{pc} + 4$ 。

==>跳转指令

(15) j addr

000010	addr (26 位)				
--------	--------------------	--	--	--	--

功能: $\text{PC} \leftarrow \{\text{PC}[31:28], \text{addr}, 2' \text{b}0\}$, 无条件跳转。

说明: 由于 MIPS32 的指令代码长度占 4 个字节, 所以指令地址二进制数最低 2 位均为 0, 将指令地址放进指令代码中时, 可省掉! 这样, 除了最高 6 位操作码外, 还有 26 位可用于存放地址, 事实上, 可存放 28 位地址, 剩下最高 4 位由 pc+4 最高 4 位拼接上。

==> 停机指令

(16) halt

111111	00000000000000000000000000000000(26 位)
--------	--

功能：停机；不改变 PC 的值，PC 保持不变。

三. 实验原理

单周期 CPU 指的是一条指令的执行在一个时钟周期内完成，然后开始下一条指令的执行，即一条指令用一个时钟周期完成。电平从低到高变化的瞬间称为时钟上升沿，两个相邻时钟上升沿之间的时间间隔称为一个时钟周期。时钟周期一般也称振荡周期（如果晶振的输出没有经过分频就直接作为 CPU 的工作时钟，则时钟周期就等于振荡周期。若振荡周期经二分频后形成时钟脉冲信号作为 CPU 的工作时钟，这样，时钟周期就是振荡周期的两倍。）

CPU 在处理指令时，一般需要经过以下几个步骤：

(1) 取指令(IF)：根据程序计数器 PC 中的指令地址，从存储器中取出一条指令，同时，PC 根据指令字长度自动递增产生下一条指令所需要的指令地址，但遇到“地址转移”指令时，则控制器把“转移地址”送入 PC，当然得到的“地址”需要做些变换才送入 PC。

(2) 指令译码(ID)：对取指令操作中得到的指令进行分析并译码，确定这条指令需要完成的操作，从而产生相应的操作控制信号，用于驱动执行状态中的各种操作。

(3) 指令执行(EXE)：根据指令译码得到的操作控制信号，具体地执行指令动作，然后转移到结果写回状态。

(4) 存储器访问(MEM)：所有需要访问存储器的操作都将在这个步骤中执行，该步骤给出存储器的数据地址，把数据写入到存储器中数据地址所指定的存储单元或者从存储器中得到数据地址单元中的数据。

(5) 结果写回(WB)：指令执行的结果或者访问存储器中得到的数据写回相应的目的寄存器中。

单周期 CPU，是在一个时钟周期内完成这五个阶段的处理。



图 1 单周期 CPU 指令处理过程

MIPS 指令的三种格式：

R 类型：

31	2625	2120	1615	1110	65	0
op	rs	rt	rd	sa	funct	
6 位	5 位	5 位	5 位	5 位	6 位	

I 类型：

31	2625	2120	1615	0
op	rs	rt	immediate	
6 位	5 位	5 位	16 位	

J 类型：

31	2625	0
op	address	
6 位	26 位	

其中，

op：为操作码；

rs: 只读。为第 1 个源操作数寄存器，寄存器地址（编号）是 00000~11111，00~1F；
rt: 可读可写。为第 2 个源操作数寄存器，或目的操作数寄存器，寄存器地址（同上）；
rd: 只写。为目的操作数寄存器，寄存器地址（同上）；
sa: 为位移量（shift amt），移位指令用于指定移多少位；
funct: 为功能码，在寄存器类型指令中（R 类型）用来指定指令的功能与操作码配合使用；
immediate: 为 16 位立即数，用作无符号的逻辑操作数、有符号的算术操作数、数据加载（Load）/数据保存（Store）指令的数据地址字节偏移量和分支指令中相对程序计数器（PC）的有符号偏移量；
address: 为地址。

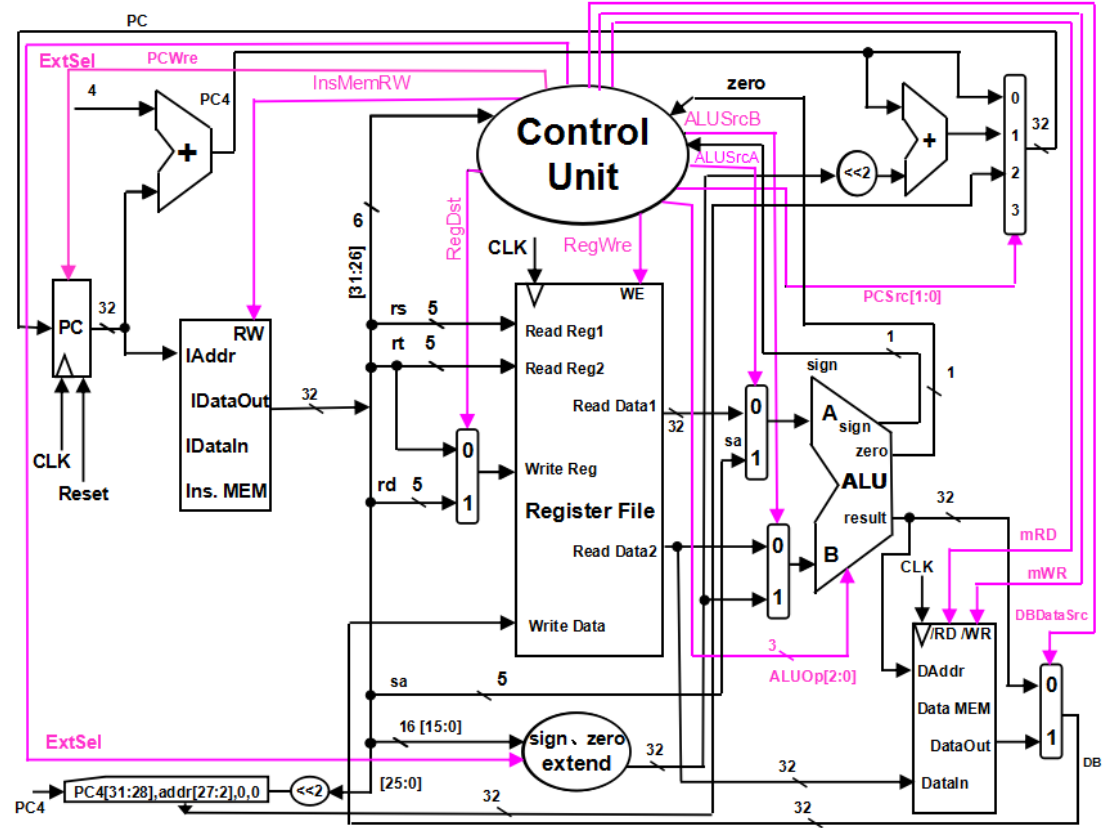


图 2 单周期 CPU 数据通路和控制线路图

图 2 是一个简单的基本上能够在单周期 CPU 上完成所要求设计的指令功能的数据通路和必要的控制线路图。其中指令和数据各存储在不同存储器中，即有指令存储器和数据存储器。访问存储器时，先给出内存地址，然后由读或写信号控制操作。对于寄存器组，先给出寄存器地址，读操作时不需要时钟信号，输出端就直接输出相应数据；而在写操作时，在 WE 使能信号为 1 时，在时钟边沿触发将数据写入寄存器。图中控制信号作用如表 1 所示，表 2 是 ALU 运算功能表。

表 1 控制信号的作用

控制信号名	状态 “0”	状态 “1”
Reset	初始化 PC 为 0	PC 接收新地址
PCWre	PC 不更改，相关指令：halt	PC 更改，相关指令：除指令 halt 外

ALUSrcA	来自寄存器堆 data1 输出，相关指令：add、sub、addiu、or、and、andi、ori、slti、beq、bne、bltz、sw、lw	来自移位数 sa，同时，进行(zero-extend)sa，即 $\{27\{1'b0\},sa\}$ ，相关指令：sll
ALUSrcB	来自寄存器堆 data2 输出，相关指令：add、sub、or、and、beq、bne、bltz	来自 sign 或 zero 扩展的立即数，相关指令：addi、andi、ori、slti、sw、lw
DBDataSrc	来自 ALU 运算结果的输出，相关指令：add、addiu、sub、ori、or、and、andi、slti、sll	来自数据存储器（Data MEM）的输出，相关指令：lw
RegWre	无写寄存器组寄存器，相关指令：beq、bne、bltz、sw、halt	寄存器组写使能，相关指令：add、addiu、sub、ori、or、and、andi、slti、sll、lw
InsMemRW	写指令存储器	读指令存储器(Ins. Data)
mRD	输出高阻态	读数据存储器，相关指令：lw
mWR	无操作	写数据存储器，相关指令：sw
RegDst	写寄存器组寄存器的地址，来自 rt 字段，相关指令：addiu、andi、ori、slti、lw	写寄存器组寄存器的地址，来自 rd 字段，相关指令：add、sub、and、or、sll
ExtSel	(zero-extend) immediate (0 扩展)，相关指令：addiu、andi、ori	(sign-extend) immediate (符号扩展)，相关指令：slti、sw、lw、beq、bne、bltz
PCSrc[1..0]	00: $pc \leftarrow pc+4$ ，相关指令：add、addiu、sub、or、ori、and、andi、slti、sll、sw、lw、beq(zero=0)、bne(zero=1)、bltz(sign=0)； 01: $pc \leftarrow pc+4+(sign-extend)immediate$ ，相关指令：beq(zero=1)、bne(zero=0)、bltz(sign=1)； 10: $pc \leftarrow \{(pc+4)[31:28],addr[27:2],2'b00\}$ ，相关指令：j； 11: 未用	
ALUOp[2..0]	ALU 8 种运算功能选择(000-111)，看功能表	

二. 相关部件及引脚说明：

Instruction Memory: 指令存储器，

Iaddr, 指令存储器地址输入端口

IDataIn, 指令存储器数据输入端口（指令代码输入端口）

IDataOut, 指令存储器数据输出端口（指令代码输出端口）

RW, 指令存储器读写控制信号，为 0 写，为 1 读

Data Memory: 数据存储器，

Daddr, 数据存储器地址输入端口

DataIn, 数据存储器数据输入端口

DataOut, 数据存储器数据输出端口

/RD, 数据存储器读控制信号，为 0 读

/WR, 数据存储器写控制信号，为 0 写

Register File: 寄存器组

Read Reg1, rs 寄存器地址输入端口
Read Reg2, rt 寄存器地址输入端口
Write Reg, 将数据写入的寄存器端口, 其地址来源 rt 或 rd 字段
Write Data, 写入寄存器的数据输入端口
Read Data1, rs 寄存器数据输出端口
Read Data2, rt 寄存器数据输出端口
WE, 写使能信号, 为 1 时, 在时钟边沿触发写入

ALU: 算术逻辑单元

result, ALU 运算结果
zero, 运算结果标志, 结果为 0, 则 zero=1; 否则 zero=0
sign, 运算结果标志, 结果最高位为 0, 则 sign=0, 正数; 否则, sign=1, 负数

表 2 ALU 运算功能表

ALUOp[2..0]	功能	描述
000	$Y = A + B$	加
001	$Y = A - B$	减
010	$Y = B \ll A$	B 左移 A 位
011	$Y = A \vee B$	或
100	$Y = A \wedge B$	与
101	$Y = (A < B) ? 1 : 0$	比较 A 与 B 不带符号
110	$Y = (((\text{rega} < \text{regb}) \ \&\& \ (\text{rega}[31] == \text{regb}[31]) \)) \ \ (\ (\text{rega}[31] == 1 \ \&\& \ \text{regb}[31] == 0) \)) \ ? \ 1 : 0$	比较 A 与 B 带符号
111	$Y = A \oplus B$	异或

需要说明的是以上数据通路图是根据要实现的指令功能的要求画出来的, 同时, 还必须确定 ALU 的运算功能(当然, 以上指令没有完全用到提供的 ALU 所有功能, 但至少必须能实现以上指令功能操作)。从数据通路图上可以看出控制单元部分需要产生各种控制信号, 当然, 也有些信号必须要传送给控制单元。从指令功能要求和数据通路图的关系得出以上表 1, 这样, 从表 1 可以看出各控制信号与相应指令之间的相互关系, 根据这种关系就可以得出控制信号与指令之间的关系表, 再根据关系表可以写出各控制信号的逻辑表达式, 这样控制单元部分就可实现了。

四. 实验器材

电脑一台, Xilinx Vivado 软件一套, Basys3板一块。

五. 实验过程与结果

CPU设计

1. ALU(算术逻辑运算单元)

```
23 module CPU1_ALU(  
24     input wire [31:0]data1,  
25     input wire [31:0]data2,  
26     input wire [2:0]ALUOp,  
27     output reg [31:0]result,  
28     output reg sign,  
29     output reg zero  
30 );  
31 always@(data1 or data2 or ALUOp) begin  
32     case(ALUOp)  
33         3'b000: result <= data1 + data2;  
34         3'b001: result <= data1 - data2;  
35         3'b010: result <= data2<<data1;  
36         3'b011: result <= data1 | data2;  
37         3'b100: result <= data1 & data2;  
38         3'b101: result <=(data1!=data2) ? 1:0;  
39         3'b110: result <=((data1<data2) && (data1[31] == data2[31] ))  
40         3'b111: result <= data1^data2;  
41         default: result <= data1^data2;  
42     endcase  
43 end  
44 always@(*)  
45 begin  
46     if(result==0) zero<=1;  
47     else zero<=0;  
48     if(result[31]==0) sign<=0;  
49     else sign<=1;  
50 end  
51 endmodule  
52  
53
```

模块输入：两个操作数，ALU控制信号。

模块输出：ALU运算结果，零信号，符号信号。

模块原理：根据ALU控制信号对两个操作数进行对应的运算（加，减，移位，按位或，按位与，无符号比较，有符号比较，异或）。若结果为0，零信号置位；若结果为负，符号信号置位。

2. SelectALUInput (ALU输入控制)


```

22 :
23 module CPU1_SelectALUInput_A(
24     input wire [31:0]Read_Data1,
25     input wire [4:0]sa,
26     input wire ALUSrcA,
27     output wire [31:0]data1
28 );
29     assign shift = {{27{1'b0}},sa};
30     assign data1=ALUSrcA?shift:Read_Data1;
31 endmodule
32

```

模块输入：寄存器堆data1输出，移位数，模块控制信号ALUSrc。

模块输出：ALU的第一个操作数。

模块原理：若ALUSrc=0，ALU第一个操作数为寄存器data1输出；否则，为扩展到32位的移位数。

```

44 :
23 module CPU1_SelectALUInput_B(
24     input wire [31:0]Read_Data2,
25     input wire ALUSrcB,
26     input wire [15:0]immediate,
27     input wire ExtSel,
28     output wire [31:0]data2
29 );
30     reg [31:0]Extend_Result;
31     always@(*)
32     begin
33         if(ExtSel==0) Extend_Result<={{16{1'b0}},immediate};
34         else
35             if(immediate[15]==0) Extend_Result<={{16{1'b0}},immediate};
36             else begin
37                 Extend_Result[31:16]<=16'b1111_1111_1111_1111;
38                 Extend_Result[15:0]<=immediate;
39             end
40         end
41     assign data2=ALUSrcB?Extend_Result:Read_Data2;
42 endmodule
43

```

模块输入：寄存器堆data2输出，立即数，模块控制信号ALUSrcB，扩展控制信号ExtSel。

模块输出：ALU的第二个操作数。

模块原理：对输入的立即数进行符号扩展(ExtSel=1)或位扩展(ExtSel=0)。若ALUSrcB=0，ALU的第二个操作数为寄存器堆data2输出；若ALUSrcB=1，ALU的第二个操作数为扩展后的立即数。

3. DataMem (数据存储器)

```
23 module CPU1_DataMem(  
24     input wire [31:0]Daddr,  
25     input wire [31:0]DataIn,  
26     input wire mRD,  
27     input wire mWR,  
28     input CLK,  
29     output wire[31:0]Datamem  
30 );  
31 reg [7:0]DataMem[0:100];  
32 always@(negedge CLK)  
33 begin  
34     if(mWR==0) ;//write  
35     else  
36         begin  
37             DataMem[Daddr+3]=DataIn[7:0];  
38             DataMem[Daddr+2]=DataIn[15:8];  
39             DataMem[Daddr+1]=DataIn[23:16];  
40             DataMem[Daddr]=DataIn[31:24];  
41         end  
42     end  
43  
44     assign Datamem[7:0]= mRD?DataMem[Daddr+3]:8'bz;  
45     assign Datamem[15:8]= mRD?DataMem[Daddr+2]:8'bz;  
46     assign Datamem[23:16]= mRD?DataMem[Daddr+1]:8'bz;  
47     assign Datamem[31:24]= mRD?DataMem[Daddr]:8'bz;  
48  
49 endmodule  
50
```

模块输入：存（取）数目标地址，存入内容，读控制信号，写控制信号。

模块输出：读数结果。

模块原理：若读控制信号=1，输出数据存储器目标地址处的数据。若写控制信号=1，将存入内容写在目标地址处。由于数据存储器内部的数据宽度为8，故需要将存入内容分段；取数时，则要将各段合并。

4. DataSelect (数据输出选择)

```
module CPU1_DataSelect(  
    input wire [31:0]result,  
    input wire [31:0]Datamem,  
    input wire DBDataSrc,  
    output wire [31:0]Data_Out  
);  
    assign Data_Out =(DBDataSrc)? Datamem : result;  
endmodule
```

模块输入：ALU运算结果，数据存储器输出，控制信号。

模块输出：数据输出Data_Out。

模块原理：若控制信号DBDataSrc=1，输出数据存储器输出；否则输出ALU运算结果。

5. InsMem (指令存储器)

```
23 module CPU1_Insmem(  
24     input wire [31:0] IAddr,  
25     input wire RW,  
26     output reg [31:0] IDataOut  
27 );  
28     reg [7:0] InsMem[0:100];  
29     initial begin  
30         $readmemb("E:/Vivado/Instructions for CPU1.txt", InsMem, 0, 100);  
31     end  
32     assign RW=1;  
33     always@(*)  
34     begin  
35         if(RW==0);  
36         else  
37             begin  
38                 IDataOut[7:0]=InsMem[IAddr+3];  
39                 IDataOut[15:8]=InsMem[IAddr+2];  
40                 IDataOut[23:16]=InsMem[IAddr+1];  
41                 IDataOut[31:24]=InsMem[IAddr];  
42             end  
43         end  
44     endmodule
```

模块输入：取指令地址，模块控制信号。

模块输出：当前要执行的指令。

模块原理：若RW=1，输出对应地址处的指令。

指令存储器的内容：

```
00100100 00000001 00000000 00001000
00110100 00000010 00000000 00000010
00000000 01000001 00011000 00100000
00000000 01100010 00101000 00100010
00000000 10100010 00100000 00100100
00000000 10000010 01000000 00100101
00000000 00001000 01000000 01000000
00010101 00000001 11111111 11111110
00101000 01000110 00000000 00000100
00101000 11000111 00000000 00000000
00100100 11100111 00000000 00001000
00010000 11100001 11111111 11111110
10101100 00100010 00000000 00000100
10001100 00101001 00000000 00000100
00100100 00001010 11111111 11111110
00100101 01001010 00000000 00000001
00000101 01000000 11111111 11111110
00110000 01001011 00000000 00000010
00001000 00000000 00000000 00110010
00000000 10000010 01000000 00100101
11111100 00000000 00000000 00000000
```

6. Register (寄存器组)

```

23 module CPU1_Register_File(
24     input CLK,
25     input wire Reset,
26     input wire RegWre,
27     input wire [4:0]rs,
28     input wire [4:0]rt,
29     input wire [4:0]rd,
30     input wire RegDst,
31     input wire [31:0]Data_Out,
32     output [31:0]Read_Data1,
33     output [31:0]Read_Data2
34 );
35
36     integer i;
37     reg [31:0] file[0:31];
38     initial begin
39         for(i=0;i<32;i=i+1)begin
40             file[i] = 32'b0000_0000_0000_0000_0000_0000_0000;
41         end
42     end
43
44     assign Read_Data1 =(rs)?file[rs]:0;
45     assign Read_Data2 =(rt)?file[rt]:0;
46
47
48     always@(negedge CLK)begin
49         if(RegWre!=0) begin
50             if(RegDst) file[rd]<=Data_Out;
51             else file[rt]<=Data_Out;
52             $display("%d", file[rd]);
53             $display("%d", file[rt]);
54             $display("%d", file[1]);
55             $display("end");
56         end
57     end
58 endmodule

```

模块输入：时钟，Reset，寄存器读写控制信号，寄存器写地址控制信号，写寄存器内容，指令的rs，rt，rd字段。

模块输出：寄存器堆输出data1和data2.

模块原理：寄存器堆的两个输出的地址分别来自指令的rs和rt字段。在时钟下降沿，若RegWre=1，写寄存器。此时，若RegDst=1，寄存器写操作的目标地址来自rd字段，否则来自rt字段。（模块最后的display语句是调试时使用的，不影响功能实现，就没有删除。）

7. PC

```

module CPU1_PC(
    input wire PCWre, //0:PC Unchanged 1:PC change
    input wire [31:0]PC_In, //The result of PC calculation multiple choice
    input wire Reset, //0:PC=0 1:PC=new
    input CLK,
    output [31:0]PC_Out //current PC
);
    reg [31:0]Current_PC; //PC Register
    // initial begin
    //     Current_PC<=0; //Initial register
    // end
    always@(posedge CLK)
    begin
        if(Reset==0) Current_PC<=0;
        else begin
            if(PCWre==0); //PCWre是1才能更新PC
            else Current_PC<=PC_In; //Change current PC
        end
    end
    assign PC_Out =Current_PC;
endmodule

```

模块输入：下一条指令地址，PC更新控制信号，Reset，时钟。

模块输出：更新后的PC。

模块原理：在时钟上升沿，若PCWre=0，PC不更新；否则，PC更新。若Reset=0，PC重置为0。

8. PCNew（下一条指令地址计算）

```

23 module CPU1_PCNew(
24     input [31:0]PC_Out,
25     input [15:0]immediate,
26     input [25:0]addr,
27     input wire [1:0]PCSrc,
28     input wire ExtSel,
29     output reg[31:0]PC_In
30 );
31 initial begin
32     PC_In=0;
33 end
34     reg [31:0]Extend_Result;
35     reg [31:0]Extend_Address;
36     reg [31:0]PC4;
37 always@(*)
38 begin
39     if(ExtSel==0) Extend_Result<={{16{1'b0}},immediate};
40 else begin
41     if(immediate[15]==0) Extend_Result<={{16{1'b0}},immediate};
42 else begin
43     Extend_Result[31:16]<=16'b1111_1111_1111_1111;
44     Extend_Result[15:0]<=immediate;
45 end
46 end
47 Extend_Address<=Extend_Result*4;
48 case(PCSrc)
49     2'b00:PC_In<=PC_Out+4;
50     2'b01:PC_In<=PC_Out+4+Extend_Address;
51     2'b10:PC_In<=32'b0000_0000_0000_0000_0000_0000_0101_0000;
52     default:PC_In<=PC_Out;
53 endcase
54 end
55 endmodule

```

模块输入：当前PC，立即数，跳转地址，PC更新控制信号PCSrc，扩展控制信号ExtSel。

模块输出：下一个PC。

模块原理：对于每一个输入的PC，若PCSrc=0，PC+4；若PCSrc=1，PC+4+立即数扩展后的结果*4；若PCSrc=2，跳转。若PCSrc=3，PC不变。

9. ControlUnit (控制单元)

```

23 module CPU1_Control_Unit(
24     input wire [5:0]op,
25     input wire [10:0]func,
26     input wire sign,
27     input wire zero,
28     output reg PCWre,
29     output reg ALUSrcA,
30     output reg ALUSrcB,
31     output reg mRD,
32     output reg mWR,
33     output reg DBDataSrc,
34     output reg ExtSel,
35     output reg [1:0]PCSrc,
36     output reg InsMemRW,
37     output reg RegDst,
38     output reg RegWre,
39     output reg [2:0]ALUOp
40 );
41
42 always@(*)
43 begin
44     case(op)
45     6'b000000:
46         case(func)
47         11'b00000_100000: begin//add
48             PCWre<=1;
49             ALUSrcA<=0;
50             ALUSrcB<=0;
51             DBDataSrc<=0;
52             RegWre<=1;
53             RegDst<=1;
54             PCSrc<=2'b00;
55             ALUOp<=3'b000;
56         end
57         11'b00000_100010:begin//sub
58             PCWre<=1;
59             ALUSrcA<=0;
60             ALUSrcB<=0;
61             DBDataSrc<=0;
62             RegWre<=1;
63             RegDst<=1;
64             PCSrc<=2'b00;
65             ALUOp<=3'b001;
66         end

```


67	⊖	11'b00000_100100: begin//and
68	○	PCWre<=1;
69	○	ALUSrcA<=0;
70	○	ALUSrcB<=0;
71	○	DBDataSrc<=0;
72	○	RegWre<=1;
73	○	RegDst<=1;
74	○	PCSrc<=2'b00;
75	○	ALUOp<=3'b100;
76	⊖	end
77	⊖	11'b00000_100101: begin//or
78	○	PCWre<=1;
79	○	ALUSrcA<=0;
80	○	ALUSrcB<=0;
81	○	DBDataSrc<=0;
82	○	RegWre<=1;
83	○	RegDst<=1;
84	○	PCSrc<=2'b00;
85	○	ALUOp<=3'b011;
86	⊖	end
87	⊖	default:begin//sll
88	○	PCWre<=1;
89	○	ALUSrcA<=1;
90	○	DBDataSrc<=0;
91	○	RegWre<=1;
92	○	RegDst<=1;
93	○	PCSrc<=2'b00;
94	○	ALUOp<=3'b010;
95	⊖	end
96	⊖	endcase
97	⊖	6'b001001:begin//addiu;
98	○	PCWre<=1;
99	○	ALUSrcA<=0;
100	○	ALUSrcB<=1;
101	○	DBDataSrc<=0;
102	○	RegWre<=1;
103	○	RegDst<=0;
104	○	ExtSel<=1;
105	○	PCSrc<=2'b00;
106	○	ALUOp<=3'b000;
107	⊖	end

108		6'b001100:begin//andi
109	○	PCWre<=1;
110	○	ALUSrcA<=0;
111	○	ALUSrcB<=1;
112	○	DBDataSrc<=0;
113	○	RegWre<=1;
114	○	RegDst<=0;
115	○	ExtSel<=0;
116	○	PCSrc<=2'b00;
117	○	ALUOp<=3'b100;
118		end
119		6'b001101:begin//ori
120	○	PCWre<=1;
121	○	ALUSrcA<=0;
122	○	ALUSrcB<=1;
123	○	DBDataSrc<=0;
124	○	RegWre<=1;
125	○	RegDst<=0;
126	○	ExtSel<=0;
127	○	PCSrc<=2'b00;
128	○	ALUOp<=3'b011;
129		end
130		6'b001010:begin//slti
131	○	PCWre<=1;
132	○	ALUSrcA<=0;
133	○	ALUSrcB<=1;
134	○	DBDataSrc<=0;
135	○	RegWre<=1;
136	○	RegDst<=0;
137	○	ExtSel<=0;
138	○	PCSrc<=2'b00;
139	○	ALUOp<=3'b110;
140		end
141		6'b101011:begin//sw
142	○	PCWre<=1;
143	○	ALUSrcA<=0;
144	○	ALUSrcB<=1;
145	○	RegWre<=0;
146	○	mWR<=1;
147	○	ExtSel<=1;
148	○	PCSrc<=2'b00;
149	○	ALUOp<=3'b000;
150		end

151	☞	6' b100011:begin//lw
152	○	PCWre<=1;
153	○	ALUSrcA<=0;
154	○	ALUSrcB<=1;
155	○	DBDataSrc<=1;
156	○	RegWre<=1;
157	○	mRD<=1;
158	○	RegDst<=1;
159	○	ExtSel<=1;
160	○	PCSrc<=2' b00;
161	○	ALUOp<=3' b000;
162	☞	end
163	☞	6' b000100:begin//beq
164	○	PCWre<=1;
165	○	ALUSrcA<=0;
166	○	ALUSrcB<=0;
167	○	RegWre<=0;
168	○	mRD<=0;
169	○	ExtSel<=1;
170	○	ALUOp<=3' b101; //
171	☞	if(zero==0)PCSrc<=2' b00;
172	☞	else PCSrc<=2' b01;
173	☞	end
174	☞	6' b000101:begin//bne
175	○	PCWre<=1;
176	○	ALUSrcA<=0;
177	○	ALUSrcB<=0;
178	○	RegWre<=0;
179	○	mRD<=0;
180	○	ExtSel<=1;
181	○	ALUOp<=3' b101; //
182	☞	if(zero==0)PCSrc<=2' b01;
183	☞	else PCSrc<=2' b00;
184	☞	end

lw	beq	bne	bltz	j	halt
1	1	1	1	1	1
0	0	0	0	X	X
1	0	0	0	X	X
1	1	1	1	1	0
1	X	X	X	X	X
1	0	0	0	X	X
1	X	X	X	X	X
X	X	X	X	X	X
1	0	0	0	X	X
X	X	X	X	X	X
1	1	1	1	X	X
000	101	101	001	X	X
00	ro=0:00 zero=1	ro=0:01 zero=0	rn=0:00 sign=1:	10	X

10. Top (CPU顶层模块)

```

23
24 module CPU1_Top(//the input and output of CPU
25     input wire CLK,
26     input wire Reset,
27     output wire PCWre,
28     output wire ALUSrcA,
29     output wire ALUSrcB,
30     output wire mRD,
31     output wire mWR,
32     output wire DBDataSrc,
33     output wire ExtSel,
34     output wire [1:0]PCSrc,
35     output wire InsMemRW,
36     output wire RegDst,
37     output wire RegWre,
38     output wire [2:0]ALUOp,
39     output wire [31:0]IDataOut,
40     output wire [31:0]PC_Out,
41     output wire zero,
42     output wire [31:0]result,
43     output wire [31:0]data1,
44     output wire [31:0]data2,
45     output wire [31:0]Read_Data1,
46     output wire [31:0]Read_Data2,
47     output wire [31:0]PC_In,
48     output wire [31:0]Data_Out,
49     output wire [4:0]rs,
50     output wire [4:0]rt,
51     output wire [4:0]rd,
52     output wire [4:0]sa,
53     output wire [5:0]op,
54     output wire [10:0]func,
55     output wire sign,
56     output wire [15:0]immediate,
57     output wire [25:0]addr
58 );

```

```

61
62 ○ assign op = IDataOut[31:26];
63 ○ assign func = IDataOut[10:0];
64 ○ assign rs = IDataOut[25:21];
65 ○ assign rt = IDataOut[20:16];
66 ○ assign rd = IDataOut[15:11];
67 ○ assign Read_Reg1 = IDataOut[25:21];
68 ○ assign Read_Reg2 = IDataOut[20:16];
69 ○ assign sa = IDataOut[10:6];
70 ○ assign immediate = IDataOut[15:0];
71 ○ assign addr = IDataOut[25:0];
72
73 CPU1_PCNew My_PCNew(
74     .immediate(immediate),
75     .addr(addr),
76     .PC_Out(PC_Out),
77     .PCSrc(PCSrc),
78     .ExtSel(ExtSel),
79     .PC_In(PC_In)
80 );
81
82 CPU1_PC My_PC(
83     .CLK(CLK), //posedge PC refresh
84     .Reset(Reset), //Reset=0, PC initialize; Reset=1, PC refresh
85     .PC_In(PC_In), //The input of PC is the output of PC choose(PC+4, PC+4+extend, PC+addr)
86     .PCWre(PCWre), //PCWre=0, halt
87     .PC_Out(PC_Out) //current PC, give to the instruction memory
88 );
89
90 CPU1_Insmem My_Insmem(
91     .IAAddr(PC_Out),
92     .RW(InsmemRW),
93     .IDataOut(IDataOut)
94 );
95

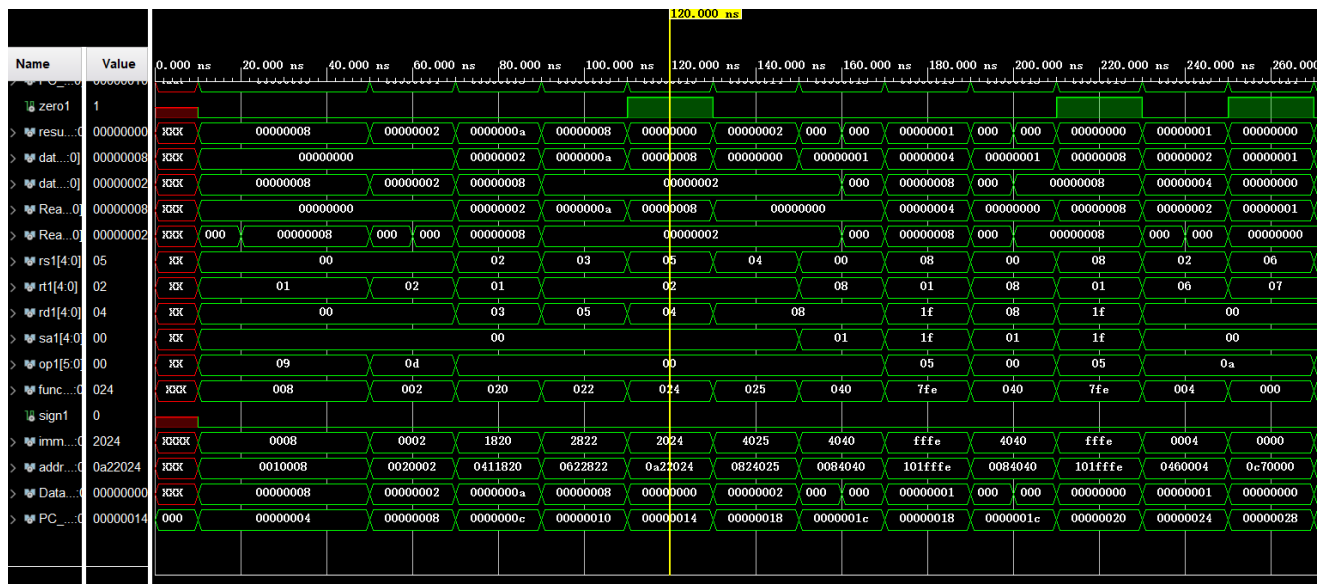
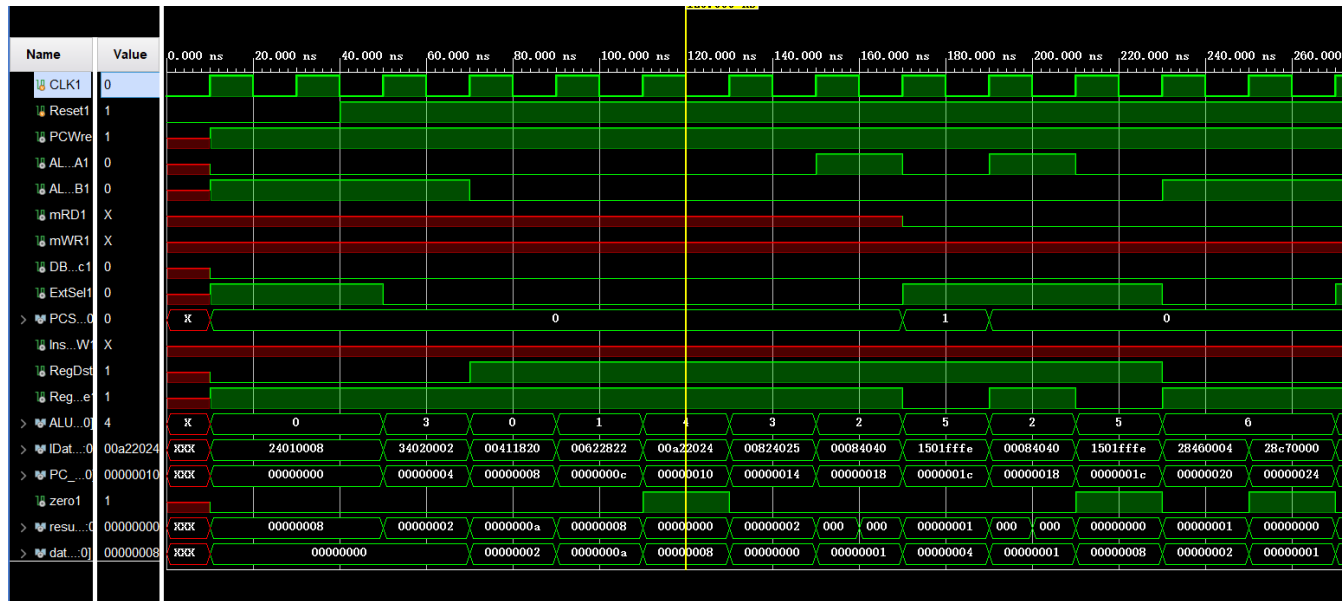
```

模块输入：CLK, Reset

模块输出：各控制信号（方便调试，上板后不体现），当前指令地址，下一条指令地址，指令的rs, rt字段，两个寄存器堆输出，ALU运算结果，数据总线输出。

模块原理：对各个模块实例化，将模块连接成一个CPU。

仿真波形：



第一条指令：0-50ns（包括Reset=0的初始阶段）

内容：addiu \$1,\$0,8

执行过程：取指令，指令译码，读出file[0]处的值，与立即数8相加，得到结果8，写入file[1]。

第二条指令：50-70ns

内容：ori \$2,\$0,2

执行过程：取指令，指令译码，读出file[0]处的值，与立即数2按位或，得到结果2，写入file[2]。

第三条指令：70-90ns

内容：add \$3,\$2,\$1

执行过程：取指令，指令译码，读出file[2]和file[1]处的值，相加，得到结果10，写入file[3]。

第四条指令：90-110ns

内容：sub \$5,\$3,\$2

执行过程：取指令，指令译码，读出file[3]和file[2]处的值，相减，得到结果8，写入file[5]。

第五条指令：110-130ns

内容：and \$4,\$5,\$2

执行过程：取指令，指令译码，读出file[5]和file[2]处的值，按位与，得到结果0，写入file[4]。

第五条指令：130-150ns

内容：or \$8,\$4,\$2

执行过程：取指令，指令译码，读出file[4]和file[2]处的值，按位或，得到结果2，写入file[8]。

第六条指令：150-170ns

内容：sll \$8,\$8,1

执行过程：取指令，指令译码，读出file[8]处的值，左移1位，得到结果4，写入file[8]。

第七条指令：170-190ns

内容：bne \$8,\$1,-2

执行过程：取指令，指令译码，读出file[8]和file[1]处的值，比较，二者不相等，跳转到地址00000018。

第八条指令：190-210ns

内容：sll \$8,\$8,1

执行过程：取指令，指令译码，读出file[8]处的值，左移1位，得到结果8，写入file[8]。

第九条指令：210-230ns

内容：bne \$8,\$1,-2

执行过程：取指令，指令译码，读出file[8]和file[1]处的值，比较，二者相等，

继续执行下一条指令。

第十条指令：230-250ns

内容：slti \$6,\$2,4

执行过程：取指令，指令译码，读出file[2]处的值，和立即数4比较，file[2]<4，ALU输出结果1，写入file[6]。

第十一条指令：250-270ns

内容：slti \$7,\$6,0

执行过程：取指令，指令译码，读出file[6]处的值，和立即数0比较，file[6]>4，ALU输出结果0，写入file[7]。



第十二条指令：270-290ns

内容：addiu \$7,\$7,8

执行过程：取指令，指令译码，读出file[7]处的值，和立即数8相加，结果为8，

写入file[7]。

第十三条指令：290-310ns

内容：beq \$7,\$1,-2

执行过程：取指令，指令译码，读出file[7]和file[1]处的值，比较，二者相等，
跳转到地址00000028

第十四条指令：330-350ns

内容：addiu \$7,\$7,8

执行过程：取指令，指令译码，读出file[7]处的值，和立即数8相加，结果为16，
写入file[7]。

第十五条指令：350-370ns

内容：beq \$7,\$1,-2

执行过程：取指令，指令译码，读出file[7]和file[1]处的值，比较，二者不相等，
继续执行下一条指令。

第十六条指令：370-390ns

内容：sw \$2,4(\$1)

执行过程：取指令，指令译码，读出file[1]处的值，和立即数4相加，得到存数
地址12，将file[2]处的值存入数据存储器的地址12处。

第十七条指令：390-410ns

内容：lw \$9,4(\$1)

执行过程：取指令，指令译码，读出file[1]处的值，和立即数4相加，得到取数
地址12，将数据存储器地址12处的值放入file[9]。

第十八条指令：410-430ns

内容：addiu \$10,\$0,-2

执行过程：取指令，指令译码，读出file[0]处的值，和立即数-2相加，得到结
果-2，写入file[10]。

第十九条指令：430-450ns

内容：addiu \$10,\$10,1

执行过程：取指令，指令译码，读出file[10]处的值，和立即数1相加，得到结
果-1，写入file[10]。

第十九条指令：450-470ns

内容: bltz \$10,-2

执行过程：取指令，指令译码，读出file[10]处的值，小于0，跳转到地址0000003C

第二十条指令：470-490ns

内容: addiu \$10,\$10,1

执行过程：取指令，指令译码，读出file[10]处的值，和立即数1相加，得到结果0，写入file[10]。

第二十一条指令：490-510ns

内容: bltz \$10,-2

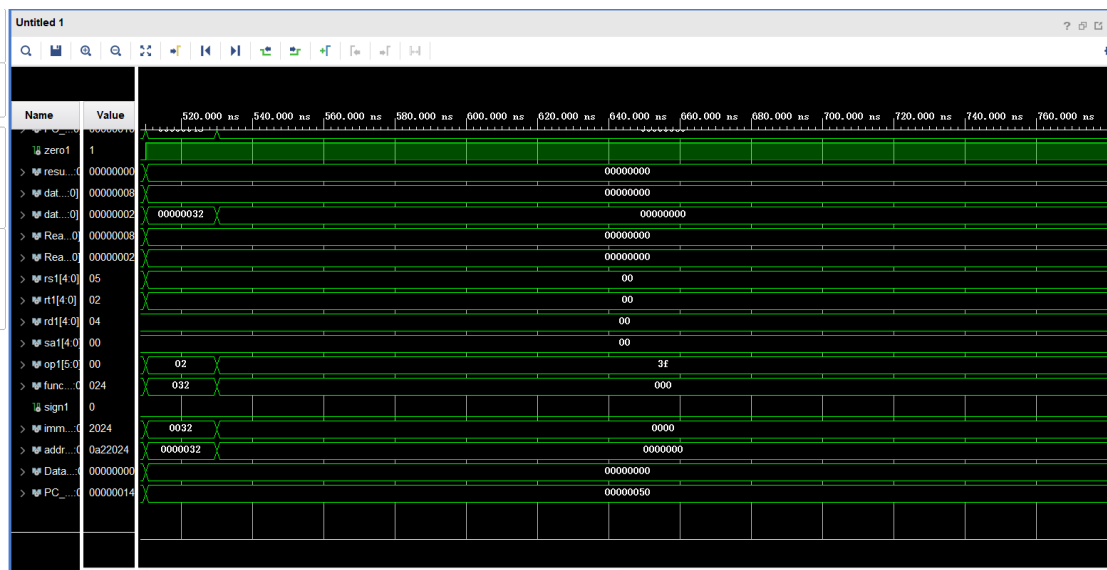
执行过程：取指令，指令译码，读出file[10]处的值，等于0，继续执行下一条指令。

第二十二条指令：510-530ns

内容: andi \$11,\$2,2

执行过程：取指令，指令译码，读出file[2]处的值，和立即数2按位与，得到结果2，写入file[11]。





第二十三条指令：530-550ns

指令内容：j 0x00000050

执行过程：取指令，指令译码，跳转到地址00000050

第二十四条指令：550ns-仿真结束

指令内容：halt

执行过程：取指令，指令译码，停机。

CPU实现（烧写到Basys3实验板）：

1. 消抖模块：Basys实验板采用机械按键，按动时可能产生抖动，故需要消抖模块，当稳定后再产生时钟脉冲。

```

module Basys_Bounce(
    input clk,
    input button,
    output signal
);
    reg [1:0]clkreg;
    reg clkslow;
    parameter times=5000;
    reg [31:0]count;

    initial begin
        clkreg=0;
        count=0;
    end

    always@(posedge clk)
    begin
        if(count<times)
            count<=count+1;
        else
            begin
                count<=0;
                clkslow=~clkslow;
            end
    end

    always@(posedge clkslow)
    begin
        clkreg[0]<=button;
        clkreg[1]<=clkreg[0];
    end

    assign signal = (clkreg[1]&clkreg[0])|(^clkreg[1]&clkreg[0]);
endmodule

```

2. 显示选择模块：根据SW的不同值，选择在数码管上显示的内容。SW=00，显示当前指令地址和下一条指令地址；SW=01或10，显示寄存器和对应的值；SW=11，显示ALU运算结果和数据输出。

```

23 module CPU2_Display(
24     input signal,
25     input [1:0]SW,
26     input [3:0]ctr,
27     input [31:0]PC_Out,
28     input [31:0]PC_In,
29     input [4:0]rs,
30     input [4:0]rt,
31     input [31:0]Read_Data1,
32     input [31:0]Read_Data2,
33     input [31:0]Data_Out,
34     input [31:0]result,
35     output reg[3:0]digit
36 );
37     always@(signal)
38     begin
39         case(ctr)
40             4'b1110:
41                 begin
42                     case(SW)
43                         2'b00:digit <= PC_In[3:0];
44                         2'b01:digit <= rs[3:0];
45                         2'b10:digit <= rt[3:0];
46                         2'b11:digit <= Data_Out[3:0];
47                     endcase
48                 end
49             4'b1101:
50                 begin
51                     case(SW)
52                         2'b00:digit <= PC_In[7:4];
53                         2'b01:digit <= {3'b000,rs[4]};
54                         2'b10:digit <= {3'b000,rt[4]};
55                         2'b11:digit <= Data_Out[7:4];
56                     endcase
57                 end
58             4'b1011:
59                 begin
60                     case(SW)
61                         2'b00:digit <= PC_Out[3:0];
62                         2'b01:digit <= Read_Data1[3:0];
63                         2'b10:digit <= Read_Data2[3:0];
64                         2'b11:digit <= result[3:0];
65                     endcase
66                 end
67             4'b0111:
68                 begin
69                     case(SW)
70                         2'b00:digit <= PC_Out[7:4];
71                         2'b01:digit <= Read_Data1[7:4];
72                         2'b10:digit <= Read_Data2[7:4];
73                         2'b11:digit <= result[7:4];
74                     endcase
75                 end
76             endcase
77         end
78     endmodule

```

3. 数码管译码显示模块：根据输入的数码管显示内容和数码管译码表，进行译码。

```

23 module Basys_Led(
24     input reset,
25     input [3:0]digit,
26     output reg[7:0]light
27 );
28 always@(digit or reset)
29 begin
30     if(reset==0) light=8'b11111110;//.
31     else begin
32         case(digit)
33             4'b0000:light=8'b00000011; //0
34             4'b0001:light=8'b10011111; //1
35             4'b0010:light=8'b00100101; //2
36             4'b0011:light=8'b00001101; //3
37             4'b0100:light=8'b10011001; //4
38             4'b0101:light=8'b01001001; //5
39             4'b0110:light=8'b01000001; //6
40             4'b0111:light=8'b00011111; //7
41             4'b1000:light=8'b00000001; //8
42             4'b1001:light=8'b00011001; //9
43             4'b1010:light=8'b00010001; //A
44             4'b1011:light=8'b11000001; //b
45             4'b1100:light=8'b01100011; //C
46             4'b1101:light=8'b10000101; //d
47             4'b1110:light=8'b01100001; //E
48             4'b1111:light=8'b01110001; //F
49             default:light=8'b11111100; //-
50         endcase
51     end
52 end
53
54 endmodule
55
56

```

4. Basys顶层模块:

模块输入: Reset, button, CLK

模块输出: SW, 数码管显示信号, 数码管控制信号。


```

23 module Basys_CPU(
24     input button,
25     input [1:0]SW,
26     input CLK,    //Basys3's Clock
27     input Reset,
28     output [7:0]light,
29     output reg[3:0]ctr
30 );
31     wire [31:0]PC_Out;
32     wire [31:0]PC_In;
33     wire [31:0]result;
34     wire [31:0]Read_Data1;
35     wire [4:0]rs;
36     wire [4:0]rt;
37     wire[31:0]IDataOut;
38     wire [31:0]Data_Out;
39     wire [31:0]Read_Data2;
40     wire signal;
41
42     parameter T1MS = 5000;
43     reg [31:0]count;
44     wire [3:0]digit;
45
46     CPU1_Top cpu(
47         .CLK(signal),
48         .Reset(Reset),
49         .IDataOut(IDataOut),
50         .rs(rs),
51         .rt(rt),
52         .PC_Out(PC_Out),
53         .Data_Out(Data_Out),
54         .result(result),
55         .PC_In(PC_In),
56         .Read_Data1(Read_Data1),
57         .Read_Data2(Read_Data2)
58     );
59
60     Basys_Bounce bounce(
61         .clk(CLK),
62         .button(button),
63         .signal(signal)
64     );
65
66     always@(posedge CLK)
67     begin
68         if(Reset==0) begin
69             count<=0;
70             ctr<=4'b0000;
71         end
72         else begin
73             count=count+1;
74             if(count==T1MS)begin
75                 count<=0;
76                 case(ctr)
77                     4'b1110:ctr <= 4'b1101;
78                     4'b1101:ctr <= 4'b1011;
79                     4'b1011:ctr <= 4'b0111;
80                     4'b0111:ctr <= 4'b1110;
81                     4'b0000:ctr <= 4'b0111;
82                 endcase
83             end
84         end
85     end
86
87     Basys_Led My_led(
88         .reset(Reset),
89         .digit(digit),
90         .light(light)
91     );

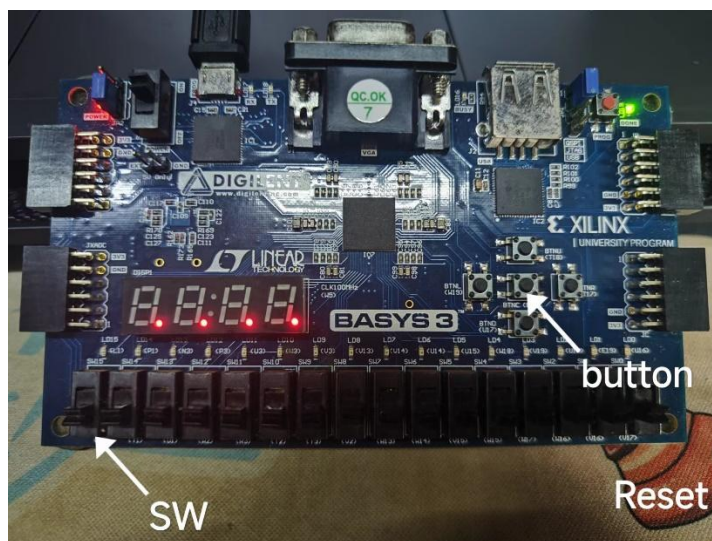
```

```

96      / ,
97
98      Basys_Display My_Display(
99          .signal(signal),
100         .ctr(ctr),
101         .SW(SW),
102         .digit(digit),
103         .PC_In(PC_In),
104         .result(result),
105         .PC_Out(PC_Out),
106         .rs(rs),
107         .rt(rt),
108         .Data_Out(Data_Out),
109         .Read_Data1(Read_Data1),
110         .Read_Data2(Read_Data2)
111     );
112 endmodule
113
114

```

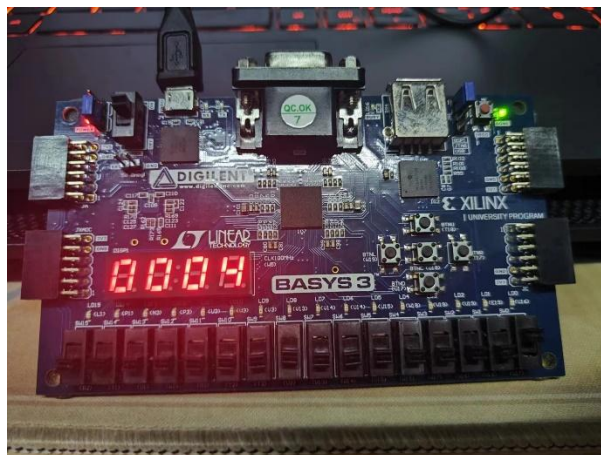
端口映射：



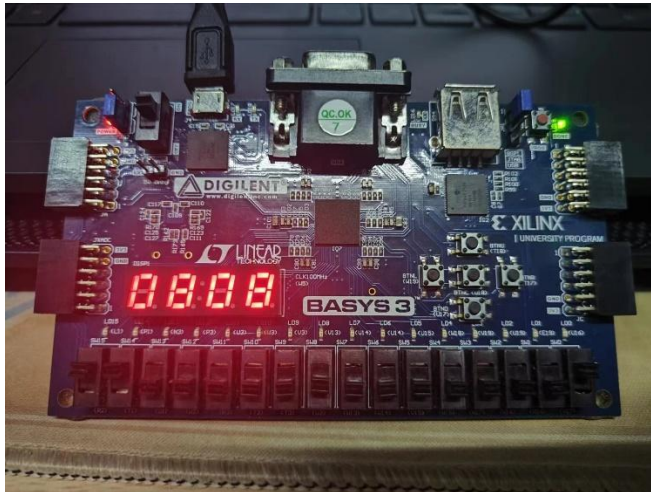
操作方法：按下button，产生一个脉冲，对应一个时钟周期。拨动SW，调整数码管显示内容。拨动Reset，重置CPU。

第一条指令：addiu \$1,\$0,8

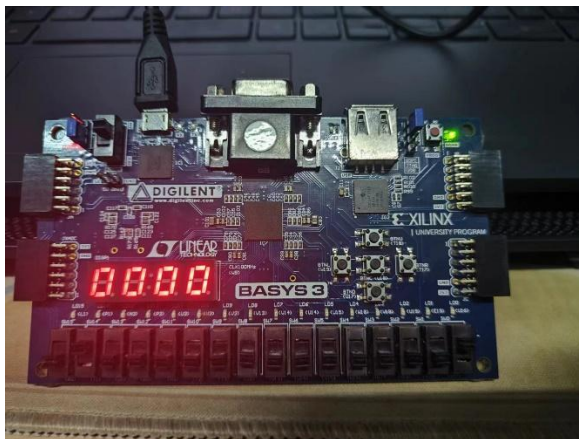
当前指令地址：00 下一条指令地址：04



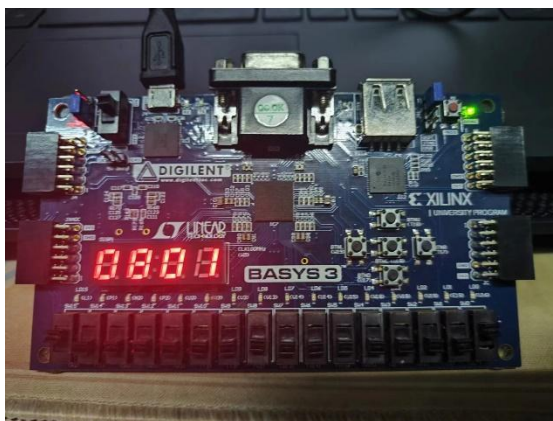
ALU运算结果：8 数据输出：8



0号寄存器的值为0.

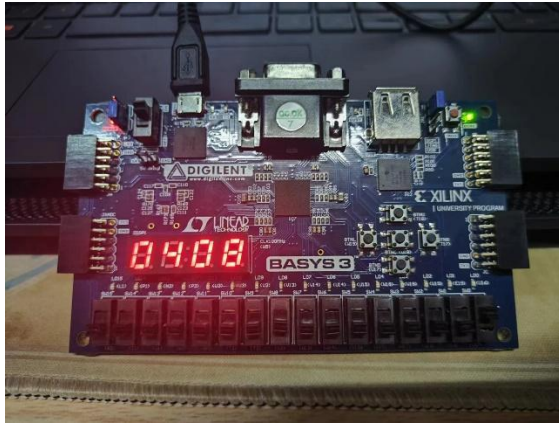


结果写入1号寄存器，1号寄存器的值为8.

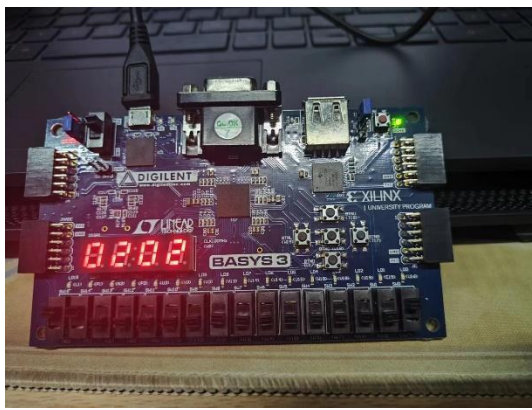


第二条指令：ori \$2,\$0,2

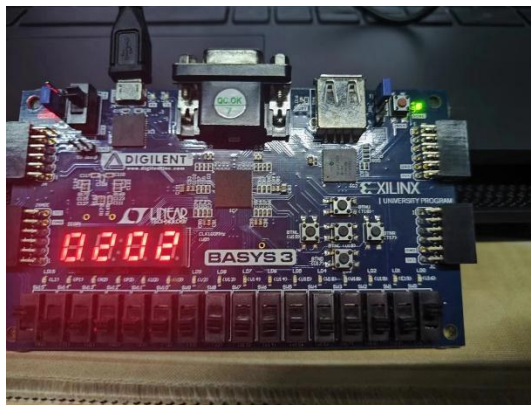
当前指令地址：04 下一条指令地址：08



ALU运算结果和数据输出均为2.

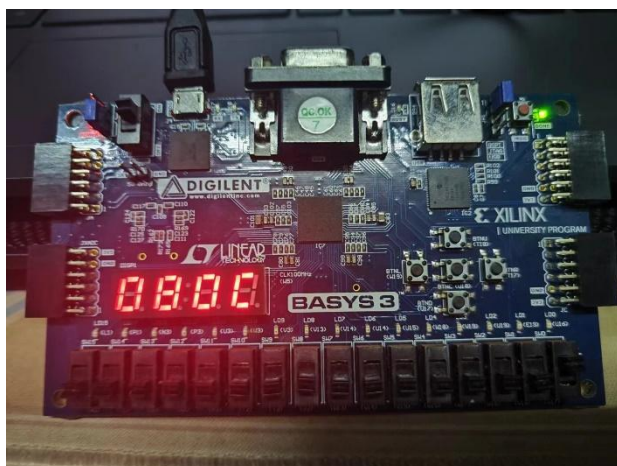


结果写入2号寄存器，2号寄存器的值为2

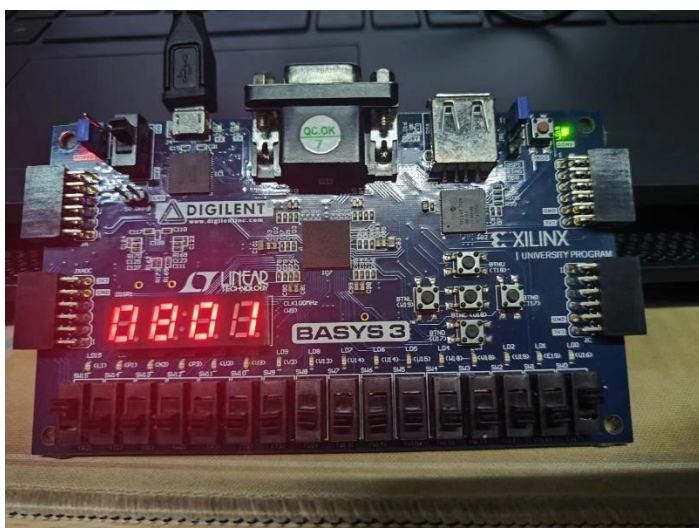


第三条指令: `add $3,$2,$1`

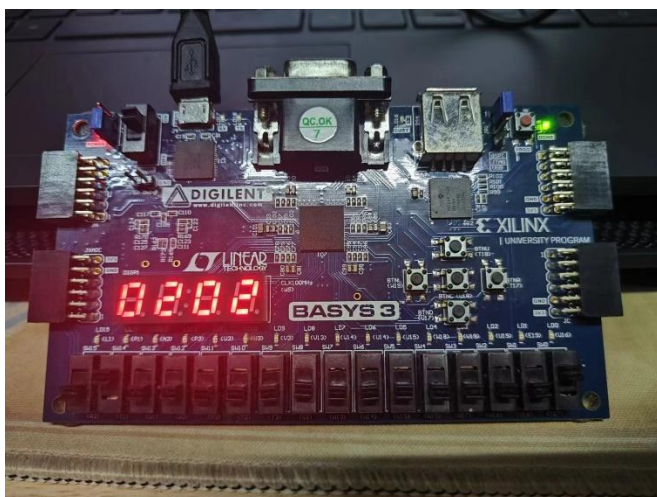
当前指令地址: 08 下一条指令地址: 0C



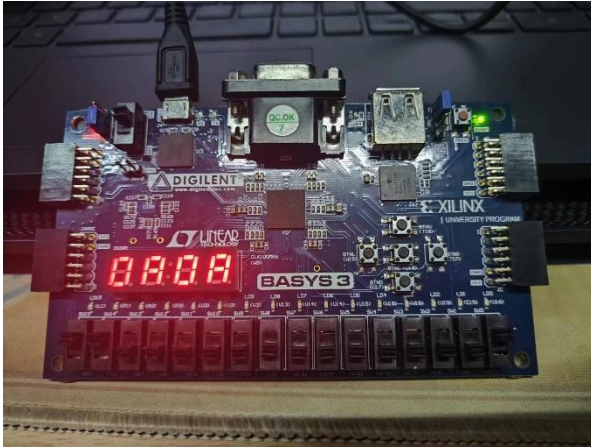
1号寄存器的值为8



2号寄存器的值为2

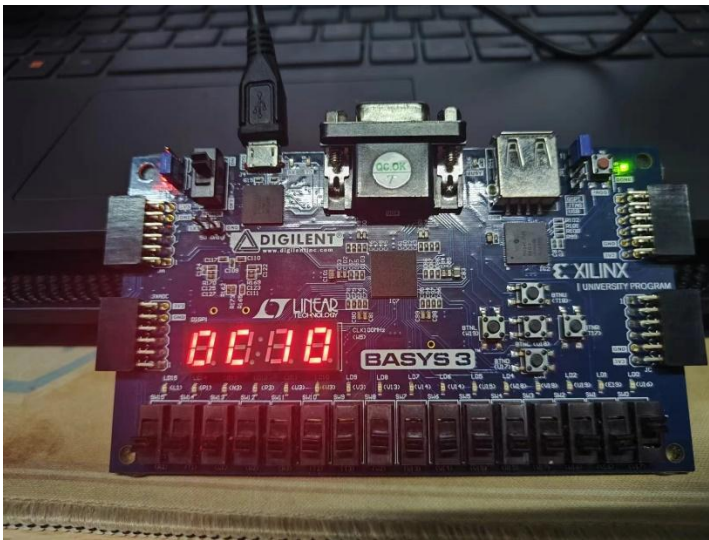


ALU运算结果和数据输出为10，写入3号寄存器。

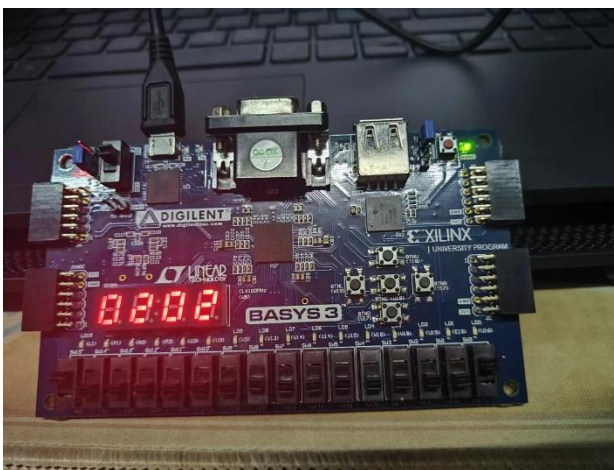


第四条指令: `sub $5,$3,$2`

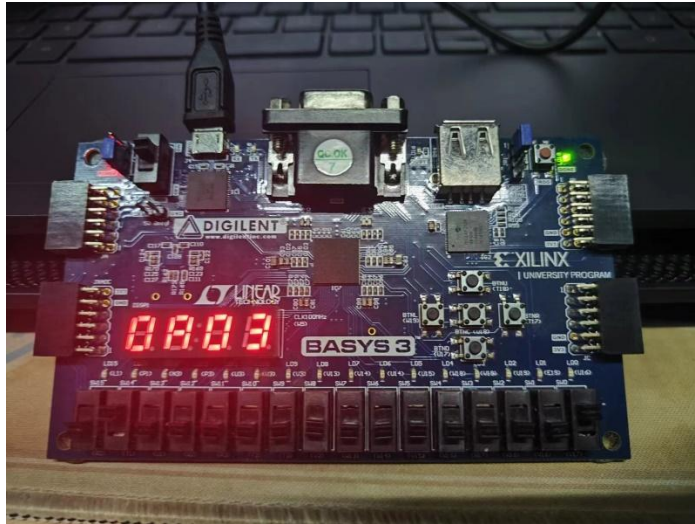
当前指令地址为0C, 下一条指令地址为10



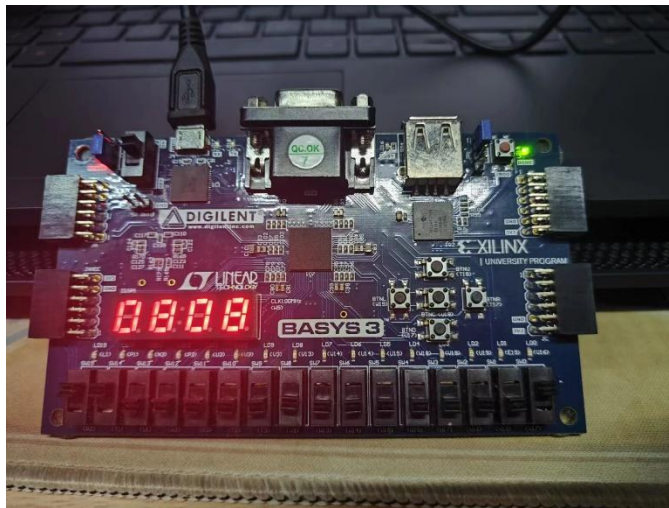
2号寄存器的值为2



3号寄存器的值为10.

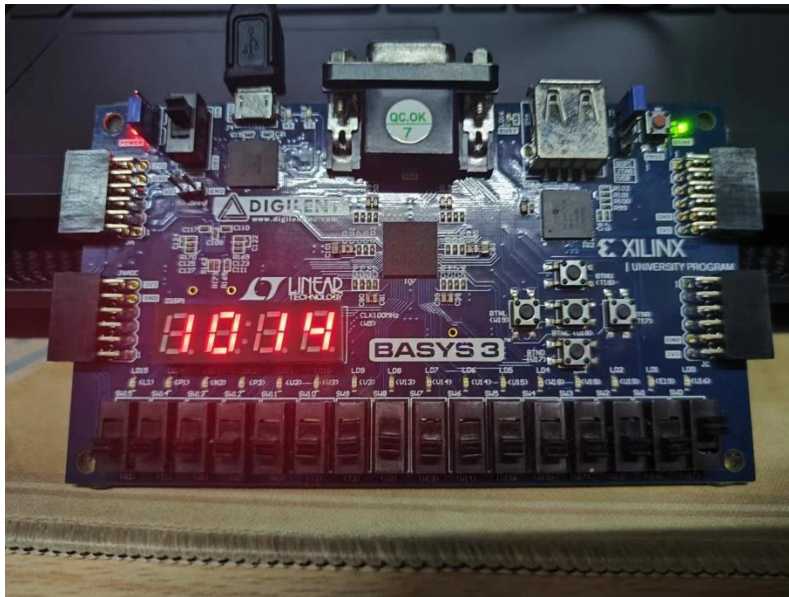


ALU运算结果和数据输出都是8，写入5号寄存器。

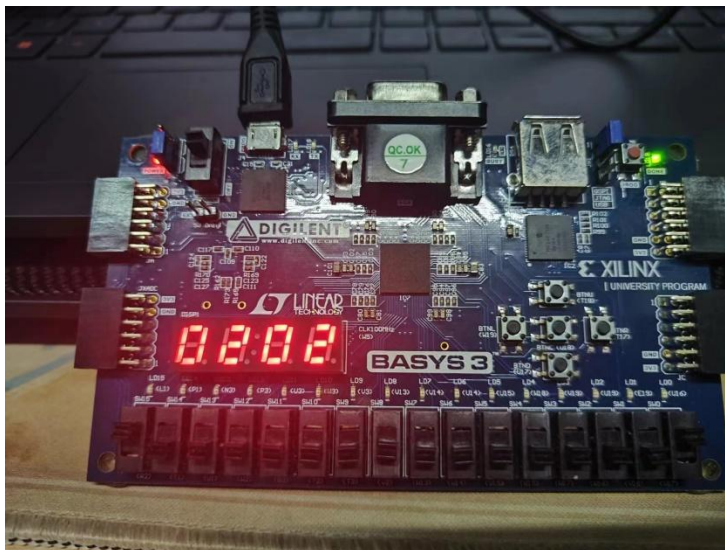


第五条指令：and \$4,\$5,\$2

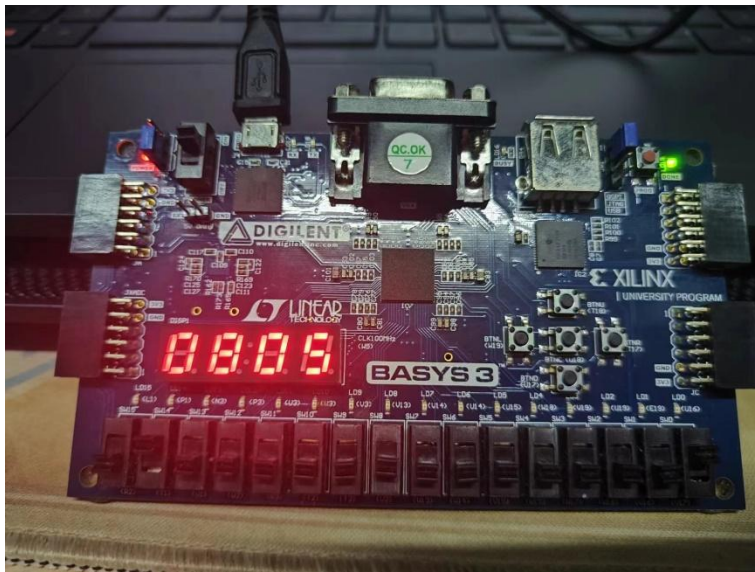
当前指令地址：10 下一条指令地址：14



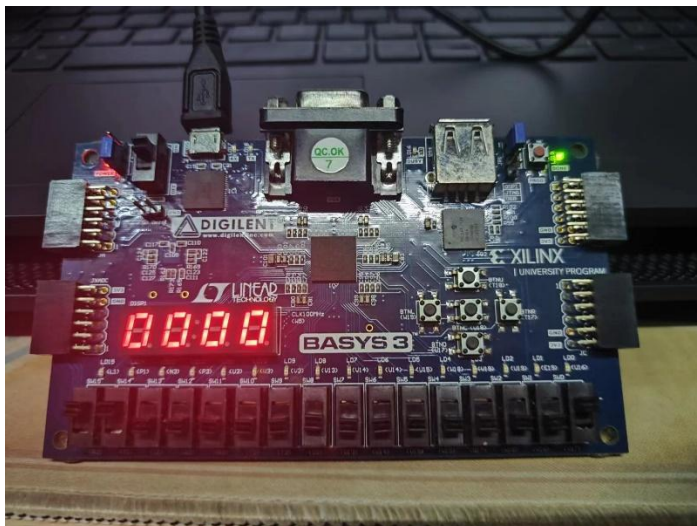
2号寄存器的值是2.



5号寄存器的值是8.

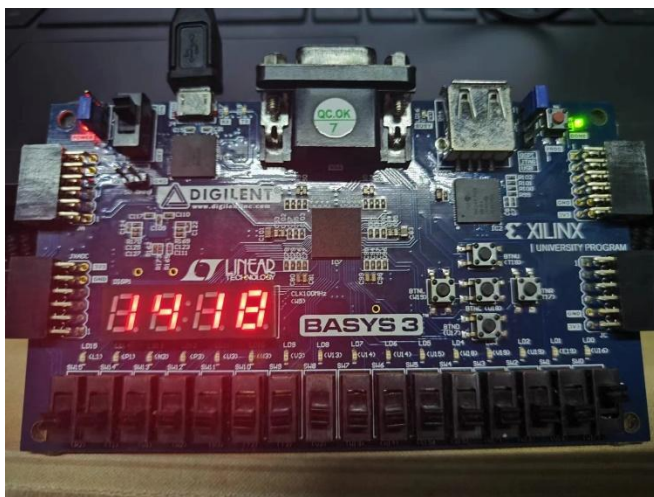


ALU运算结果和数据输出都为0，写入4号寄存器。

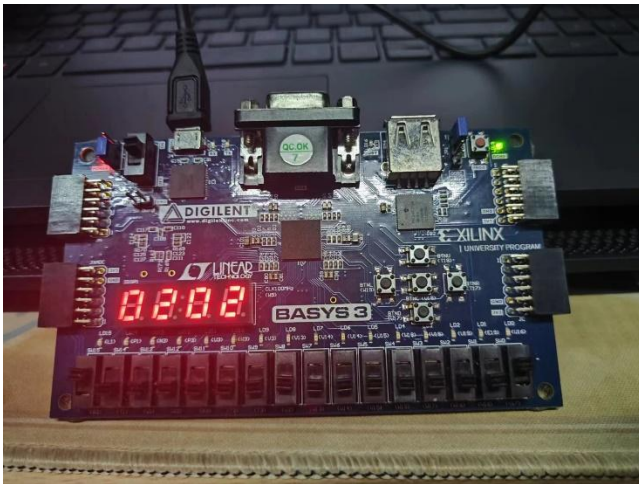


第六条指令：or \$8,\$4,\$2

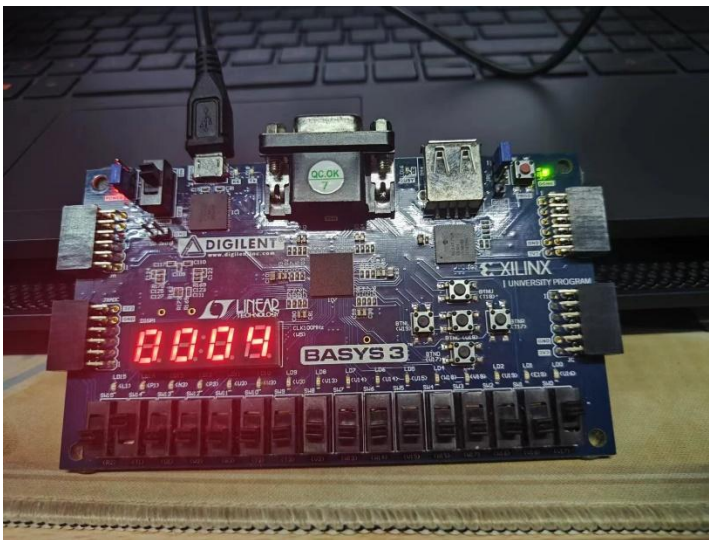
当前指令地址：14 下一条指令地址：18



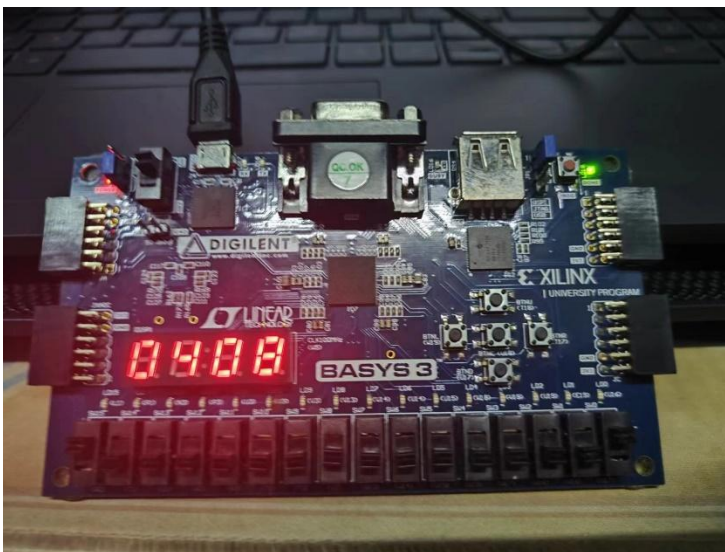
2号寄存器的值是2



4号寄存器的值是0.



结果写入8号寄存器，8号寄存器的值是4.



六、心得体会

实现单周期CPU，是我进入大学以来完成的最困难的一项任务。它的主要困难点不在于实验本身的难度，而是第一次做大作业时经验不足走的弯路。

首先，由于不熟悉vivado和verilog，我在一开始写代码的阶段经常遇到各种error，需要一遍遍翻看老师的课件和资料。在运行仿真的时候，我没有看warning的意识，导致被一些简单的问题困扰了很久。

其次，在我写寄存器组模块的时候，遇到了软件问题。我写的寄存器组从代码来看没有任何问题，但是却不能正常运行。我百思不得其解，寻求助教老师的帮助。在助教老师的帮助下，最终在新建了一个工程并将原来的模块全部添加进去后，CPU可以正常运行了。通过这次经历，我以后在做大作业遇到问题无法解决时，会考虑在别人的电脑上运行程序，从而排除电脑和软件的问题。

最后，我在做作业的过程中，也积累了很多debug的经验。比如，在出现某个整体性问题不知道如何解决的时候，可以尝试重建工程、复现bug，从而确定问题出在哪里。

感谢助教老师在实验过程中的指导和帮助。通过进行单周期CPU实验，我对CPU的原理有了更深刻的了解，增长了debug和做大作业的经验，提升了自身能力。